

Architecture du RAG — chatbot_ms

Ghabetna

3 septembre 2026 — révisé le 4 septembre 2026

Ce document explique, étape par étape, comment fonctionne l’assistant conversationnel de Ghabetna : ce qui est réellement implémenté dans **rag/** (avec les fichiers et fonctions exacts), pourquoi chaque choix a été fait, quelles étaient les alternatives possibles, et — en fin de document — quelles métriques manquent encore pour évaluer objectivement la qualité du système.

Écrit pour deux usages : comprendre le système en profondeur (utile en soutenance), et servir de base avant d’ajouter les métriques d’évaluation au code.

*Révision du 4 septembre 2026 : ajout de la section 5.4 sur l’étanchéité entre spécialités (filtrage par domaine), absente de la version initiale bien que le mécanisme existe dans le code depuis le début du projet. Point déclencheur : l’ajout des corpus **camper** et **apiculture** en septembre 2026, qui a permis de vérifier ce mécanisme en pratique sans toucher au code.*

Contents

1	Vue d’ensemble du pipeline	2
2	Étape 1 — Chunking (rag/chunker.py)	2
3	Étape 2 — Embedding (rag/indexer.py, fonction embedder)	3
4	Étape 3 — Index lexical BM25 (rag/indexer.py, fonction indexer_bm25)	6
5	Étape 4 — Retrieval hybride (rag/retriever.py)	6
5.1	Recherche dense (_dense)	6
5.2	Recherche lexicale (_bm25)	7
5.3	Fusion — Reciprocal Rank Fusion (_rrf)	8
5.4	Étanchéité entre spécialités — filtrage par domaine	8
6	Calcul déterministe des dates (rag/calendrier.py)	10
7	Génération (rag/generator.py)	11
8	Cache de réponses (rag/cache.py)	13
9	Ce qui manque : métriques d’évaluation du RAG	13
9.1	Métriques de retrieval	13
9.2	Métriques de génération	14
9.3	Évaluation de bout en bout	14
9.4	Ce qu’il faudrait construire avant de coder les métriques	14

1 Vue d'ensemble du pipeline

Le pipeline se déroule en deux temps.

Un **job hors-ligne** (`scripts/build_index.py`), joué seulement quand le corpus change :

1. **Chunking** — `rag/chunker.py` → `chunks.json`
2. **Embedding** — `rag/indexer.py` → `vecteurs.npy`
3. **Index BM25** — `rag/indexer.py` → `bm25.pkl`

À chaque question, avec un modèle déjà chargé en mémoire (`app/main.py`, `lifespan`) :

4. **Retrieval hybride** (dense + BM25 + RRF, **filtré par domaine**) — `rag/retriever.py` → top-8 chunks
 ↪ `rag/calendrier.py` — si la question porte sur une espèce et une date, un verdict est calculé
5. **Génération** — `rag/generator.py` → appel Groq (LLM)
6. **Cache** — `rag/cache.py` → réponse persistée sur disque

La réponse, ses sources et le verdict éventuel sont renvoyés par `app/main.py` (POST `/api/chat/`).

2 Étape 1 — Chunking (`rag/chunker.py`)

Ce qui est fait

- **Un chunk = un article de loi**, détecté par un motif regex sur le titre (`RE_ARTICLE`), pas par le niveau de titre Markdown (`##/###`) — celui-ci varie d'un texte à l'autre (le code forestier met ses articles en `##` pour le Titre I et en `###` pour le Titre II).
- **Découpage sémantique multi-domaines** : le nom du sous-dossier de `data/corpus/` devient la métadonnée `domaine`. Ajouter un nouveau domaine = déposer des `.md` dans un nouveau dossier, aucune ligne de code à changer (`chunker_corpus()`).
- **Sous-découpage des tableaux** (`ecarter_tableau`) : un tableau Markdown est éclaté **ligne par ligne**, chaque ligne devenant son propre chunk. Raison concrète : le tableau de l'article premier de l'arrêté de chasse mélange 10 espèces (lièvre, sanglier, bécasse...) dans un seul bloc de texte — un vecteur d'embedding unique ne peut pas représenter 10 régimes de dates différents à la fois.
- **Sous-découpage des articles trop longs** (`sous_decouper`) : au-delà de 1500 caractères (`config.TAILLE_MAX_CHUNK`), découpage par paragraphe (jamais au milieu d'un paragraphe, jamais au milieu d'un tableau).
- **Double préfixe contextuel** — décision la plus importante de ce module :

- **prefixe** (long, ~25 tokens) : source complète + hiérarchie (Titre/Chapitre) + titre. Utilisé pour **BM25 et l’affichage**.
- **prefixe_court** (~6 tokens) : « *Arrêté chasse 2025/2026, Article 12* ». Utilisé **uniquement pour l’embedding**.
- **Pourquoi deux préfixes** : un vecteur d’embedding a une taille fixe où chaque token compte. Le préfixe long, répété à l’identique sur 57 chunks de l’arrêté, représentait la moitié du contenu d’un chunk court — tous les vecteurs se ressemblaient. C’est ce qui faisait remonter l’article 10 (colportage) en tête de questions sans rapport. BM25, lui, pondère par IDF : les mots répétés sur tout le corpus ont un poids quasi nul automatiquement, donc le préfixe long ne le pénalise pas.
- **Fidélité juridique vs recherche** (`extraire_alias_sic`) : quand le texte officiel contient une coquille (ex. article 176 écrit « *chasseurs* »), le texte affiché garde la faute (fidélité au texte de loi), mais un alias correct (« *chasseurs* ») est ajouté aux métadonnées pour que BM25 le retrouve quand même.
- **Séparation normatif / avertissement / note** (`separer_contenu`) : les blocs > ****Avertissement de concordance**** (renvois périmés dans le texte officiel) sont indexés et signalés au LLM ; les blocs > ****Note de fidélité**** sont purement éditoriaux et exclus de l’index.

Alternative considérée : découpage par taille fixe (sliding window)

Critère	Découpage par article (choisi)	Découpage par taille fixe
Cohérence sémantique	Un chunk = une unité juridique complète	Peut couper une règle en deux, brisant le sens
Traçabilité (citation d’article)	Immédiate — chaque chunk connaît son article	Nécessite de retrouver l’article a posteriori
Complexité d’implémentation	Plus élevée (regex, exceptions : tableaux, sous-sections géographiques)	Plus simple, générique
Adapté à un corpus juridique structuré	Oui	Non — ignore la structure

Le découpage par article a été retenu car le corpus est un texte **juridique structuré** où l’unité de sens (l’article) est bien définie et doit rester citable telle quelle.

3 Étape 2 — Embedding (`rag/indexer.py`, fonction `embedder`)

Ce qui est fait

- **Modèle** : BAAI/bge-m3 (`config.MODELE_EMBEDDING`), via `sentence-transformers`, dimension **1024**.
- **Aucun préfixe passage:/query** : n’est ajouté au texte avant embedding pour ce modèle (`config.PREFIXE_PASSAGE = ""`) — bge-m3 n’en a pas besoin, contrairement à `intfloat/multilingual-e5` qui les exige.

- **Cache par empreinte** (`_empreinte_corpus`) : un hash SHA-256 du modèle + préfixe + contenu de chaque chunk est comparé au cache existant (`vecteurs.meta.json`). Si rien n'a changé, les vecteurs sont rechargés depuis `vecteurs.npy` au lieu d'être recalculés — l'embedding complet prend ~11 minutes sur CPU, le rechargement quelques secondes.
- **Normalisation** (`normalize_embeddings=True`) : chaque vecteur a une norme de 1, ce qui rend le **produit scalaire mathématiquement égal au cosinus** — `store.py` peut donc chercher par une simple multiplication matricielle, sans calcul de norme à chaque requête.

Choix du modèle : comparaison

Modèle	Dim.	Multi. FR/AR	Préfixe requis	Pour recherche / écart
BAAI/bge-m3 (retenu)	1024	Oui (100+ langues)	Non	Bon com- pro- mis qual- ité/c en lo- cal, pas de dépe danc à une clé API pour l'embl
intfloat/multilingual-e5-base	768	Oui	Oui (query:/passage:)	Dimen plus faible → moins de nu- ance risqu d'err si- len- cieus si le pré- fixe est ou- blié
paraphrase-multilingual-MiniLM-L12-v2	384	Oui (moindre qualité)	Non	Plus rapide mais di- men- sion trop faible

Règle opérationnelle notée dans le code (`config.py:41-44`) : changer de modèle d’embedding oblige à **tout réindexer** — deux modèles produisent des espaces vectoriels incomparables, un mélange romprait totalement la recherche, **silencieusement** (aucune erreur, juste des résultats incohérents).

4 Étape 3 — Index lexical BM25 (`rag/indexer.py`, fonction `indexer_bm25`)

Ce qui est fait

- Bibliothèque `rank_bm25` (`BM25Okapi`), construit sur le texte `texte_indexe` (préfixe long + corps + alias de coquilles).
- **Tokenisation dédiée** (`tokeniser`) : minuscules + suppression des accents (NFD + filtrage des marques diacritiques), pour que « *Aïn Drahm* » et « *Ain Drahm* » produisent les mêmes tokens.

Pourquoi BM25 en plus du dense

Le corpus contient des éléments que l’embedding sémantique gère mal :

- ~800 toponymes dans l’article 12 (noms de réserves de chasse)
- des termes rares sans voisin sémantique clair (« *slougui* », « *chevrotine* », « *adjudicataires* »)
- des montants exacts que le dense ne distingue pas (« *90 dinars* » vs « *150 dinars* »)

À l’inverse, sur « *puis-je tirer un sanglier la nuit* », le dense retrouve l’article 172 qui ne contient aucun de ces mots — BM25 seul échouerait. D’où la fusion des deux (section suivante).

Alternative considérée : dense uniquement

Un système dense-only est plus simple à maintenir (un seul index), mais le test de retrieval documenté dans `scripts/test_retrieval.py` montre concrètement l’échec sur des requêtes lexicales précises (montants, toponymes, mots rares). La complexité additionnelle de BM25 + fusion a été jugée nécessaire au vu de la nature du corpus (texte juridique avec beaucoup de données factuelles précises).

5 Étape 4 — Retrieval hybride (`rag/retriever.py`)

5.1 Recherche dense (`_dense`)

Encode la question (avec `PREFIXE_QUERY`, vide pour `bge-m3`), puis interroge `NumpyStore.query()`. **Magasin de vecteurs = un tableau NumPy en mémoire**, pas de base vectorielle dédiée (`rag/store.py`). La recherche est un produit matriciel `vecteurs @ question` : ~0,1 ms pour ~170-500 chunks.

Choix du magasin de vecteurs : comparaison

Solution	Latence à cette échelle	Complexité opérationnelle	Justification
NumPy brute-force (retenu)	$\sim 0,1$ ms	Aucune (aucun service à déployer)	À cette échelle (170-500 chunks $\times$ 1024 dim $\sim 0,7$ -2 Mo), un calcul exact est plus rapide qu'un index approximatif, sans complexité supplémentaire
FAISS (index HNSW)	Comparable ou plus lent en dessous de $\sim 100k$ vecteurs	Dépendance native compilée à gérer	Utile seulement à grande échelle — non atteinte ici
Base vectorielle managée (Pinecone, Qdrant Cloud)	Latence réseau (dizaines de ms)	Dépendance externe, coût récurrent	Ajoute une dépendance réseau pour un gain nul à ce volume de données
pgvector (extension PostgreSQL)	Correcte	Nécessite une base dédiée + requêtes SQL	Pertinent si le corpus grossissait fortement ; non nécessaire ici

Le commentaire du code (`store.py:21-23`) formalise l'argument clé : la recherche vectorielle représente **0,005 % du temps total** d'une requête (0,1 ms sur 2 à 5 secondes d'appel LLM) — ce n'est jamais le goulot d'étranglement, donc l'optimiser au-delà du nécessaire n'aurait aucun effet perceptible.

5.2 Recherche lexicale (`_bm25`)

Expansion de requête (`rag/synonymes.py`, fonction `enrichir`) appliquée **uniquement côté BM25** : un dictionnaire fait le pont entre le vocabulaire du citoyen (« *prix* », « *risque* ») et celui du

texte officiel (« *redevance domaniale* », « *puni de...emprisonnement* »). L'expansion **concatène** les synonymes à la question plutôt que de remplacer les mots d'origine — ceux-ci gardent tout leur poids, les synonymes ne font qu'ouvrir des portes supplémentaires.

Cette expansion n'est **volontairement pas appliquée au dense** : ajouter des mots à une requête avant de l'embedder déplacerait son vecteur vers une moyenne floue, ce qui dégraderait le résultat au lieu de l'améliorer (l'embedding capture déjà les synonymes sémantiquement).

5.3 Fusion — Reciprocal Rank Fusion (_rrf)

- Les deux classements (dense, BM25) ont des scores **non comparables** (cosinus borné [0,1] vs score BM25 non borné, dépendant du corpus).
- RRF ignore les scores et ne garde que les **rangs** :

$$score(chunk) = \sum \frac{1}{k + rang}$$

avec $k = 60$ (constante standard de la littérature, amortit l'écart entre les tout premiers rangs).

- Un chunk 1^{er} en dense **et** 3^e en BM25 remonte devant un chunk 1^{er} en dense seulement — ce que valident les deux méthodes est privilégié, sans écarter ce qu'une seule des deux trouve.
- **Déduplication** sur les 200 premiers caractères du texte (deux chunks identiques ne doivent pas occuper deux places du top-k).

Alternative considérée : re-ranking par cross-encoder

Une architecture plus sophistiquée ajouterait une étape de **re-ranking** (ex. **cross-encoder/ms-marco-MiniLM**) après la fusion RRF, qui réévalue chaque paire (question, chunk) conjointement plutôt que par similarité de vecteurs indépendants. Cela améliore généralement la précision au prix d'une latence supplémentaire. **Non implémenté actuellement** — piste d'amélioration possible si les métriques (section 9) révèlent un besoin de précision plus fine sur le top-k.

5.4 Étanchéité entre spécialités — filtrage par domaine

Les sections 5.1 à 5.3 décrivent comment deux classements sont produits puis fusionnés — mais ne disent encore rien de **quels chunks ont le droit d'entrer dans ces classements**. Un chasseur ne doit jamais recevoir un extrait du corpus **camper** ou **apiculture**, même si son vecteur est mathématiquement le plus proche de sa question. Ce filtrage est un mécanisme à part entière, appliqué **en amont** de `_dense` et `_bm25`, et non une conséquence de la fusion RRF.

Où la spécialité devient une liste de domaines. Chaque citoyen a une spécialité unique choisie à l'inscription (**chasseur**, **campeur** ou **apiculteur**). `rag/config.py` fait le pont entre cette spécialité et les domaines du corpus :

```
SPECIALITES = {
    "chasseur":    "chasse",
    "campeur":     "camper",
    "apiculteur":  "apiculture",
}
```



```
def domaines_pour(specialite: str | None) -> list[str] | None:
    if not specialite:
        return None
    dom = SPECIALITES.get(specialite.strip().lower())
    return [dom, DOMAINE_COMMUN] if dom else [DOMAINE_COMMUN]
```

Un apiculteur reçoit donc ["apiculture", "app"] : sa réglementation propre, plus le domaine app commun à tous (aide sur l'application, création d'une alerte...). Jamais *chasse* ni *camper*. Le cas `specialite = None` renvoie `None` (aucun filtre) : un choix délibéré, réservé aux scripts de test internes (`scripts/test_retrieval.py`), jamais atteint depuis l'API publique puisque `app/main.py` calcule toujours la spécialité à partir d'un citoyen authentifié.

app/main.py calcule ce filtre une fois par requête. `domaines = config.domaines_pour(body.specialite)` est calculé au tout début du traitement de la requête, puis transmis à `Retriever.rechercher(question, domaines=domaines)`. Le filtre descend ensuite séparément dans les deux recherches — et c'est là que les deux mécanismes divergent.

Côté dense : filtrage *avant* le tri (`rag/store.py`). `NumpyStore.query()` calcule d'abord la similarité cosinus de **tous** les chunks du corpus (un seul produit matriciel), puis applique un masque booléen sur la métadonnée de domaine *avant* de sélectionner le top-k :

```
if domaines:
    autorises = set(domaines)
    masque = np.array([m.get("domaine") in autorises for m in self.metas])
    scores = np.where(masque, scores, -np.inf)
```

Un chunk hors domaine reçoit un score de $-\infty$: il ne peut plus, mathématiquement, se retrouver dans le top-k, quelle que soit sa proximité sémantique avec la question. L'étanchéité est donc garantie *avant* tout classement, et non par un filtre appliqué après coup sur un résultat déjà exposé.

Côté BM25 : filtrage *après* le tri, par nécessité (`rag/retriever.py`). La bibliothèque `rank_bm25` ne connaît aucune notion de métadonnée ni de domaine : elle renvoie un score par position dans le corpus entier, sans possibilité de filtrer en amont. `_bm25()` récupère donc d'abord les $k \times 3$ meilleurs candidats tous domaines confondus, puis élimine ceux dont le domaine ne correspond pas :

```
ordre = sorted(range(len(scores)), key=lambda i: -scores[i])[:k * 3]
...
if domaines and meta.get("domaine") not in domaines:
    continue
```

Le commentaire du code résume la raison de cette asymétrie : « *BM25 ne sait pas filtrer : on applique le domaine a posteriori* ».

Limite assumée. La marge $k \times 3$ (60 candidats bruts pour `TOP_K_BM25 = 20`) suffit en pratique, le nombre de chunks par domaine restant petit devant le corpus total (61 à 170 chunks par domaine sur 320 au total). Mais si une question renvoyait ses 60 meilleurs scores BM25 entièrement dans un autre domaine, la liste BM25 filtrée reviendrait vide pour cette requête précise — la recherche dense, filtrée en amont et donc insensible à ce problème, continuerait néanmoins à alimenter la fusion RRF.

Preuve que ce n'était pas une improvisation. Le mécanisme de filtrage par domaine — et le dictionnaire `SPECIALITES` qui l'alimente — existaient déjà dans le code avant même que les corpus `camper` et `apiculture` ne soient écrits. Le commentaire de `rag/store.py` (rédigé début août 2026, plusieurs semaines avant l'ajout de ces deux corpus) l'atteste noir sur blanc :

170 chunks x 1024 dimensions = une matrice de 700 Ko.
Avec les corpus `camper` et `beekeeper` : ~500 chunks, 2 Mo.

Ajouter les spécialités `campeur` et `apiculteur` en septembre 2026 n'a donc nécessité **aucune modification** de `store.py`, `retriever.py`, `config.py` ni `app/main.py` : uniquement le dépôt de nouveaux fichiers `.md` dans `data/corpus/camper/` et `data/corpus/apiculture/`, plus 5 lignes cosmétiques dans `chunker.py` (un dictionnaire d'affichage, voir section 2) — la même conclusion que pour le chunking multi-domaines s'applique donc aussi au filtrage par domaine.

Alternative considérée : un magasin de vecteurs séparé par domaine

Critère	Magasin partagé + filtre à la volée (retenu)	Un magasin séparé par domaine
Garantie d'étanchéité	Runtime : masque $-\infty$ (dense) / filtre a posteriori (BM25)	Structurelle : un chunk hors domaine n'existe physiquement pas dans le magasin interrogé
Ajout d'un domaine	Déposer des <code>.md</code> , zéro ligne de code — vérifié deux fois en pratique (<code>camper</code> , <code>apiculture</code>)	Créer et charger un nouveau magasin, adapter le code de chargement au démarrage
Coût mémoire au démarrage	Un seul modèle + un seul <code>vecteurs.npy</code> chargés	N magasins chargés, ou chargement paresseux à implémenter
Risque en cas d'oubli du filtre	Une requête sans <code>domaines=</code> renvoie tout le corpus (comportement volontairement permis pour les tests, jamais atteint en production)	Aucun risque équivalent — le mauvais magasin ne serait simplement jamais interrogé
Complexité d'implémentation	Faible (un paramètre optionnel propagé dans deux fonctions)	Plus élevée (gestion multi-instance de <code>Retriever</code> ou de <code>NumpyStore</code>)

Le filtrage à la volée sur un index partagé a été retenu pour la même raison que le découpage par article (section 2) : il préserve la promesse « *ajouter un domaine = déposer des fichiers, zéro ligne de code* ». Cette promesse a été formulée avant même que les domaines `camper` et `apiculture` existent, puis vérifiée deux fois en pratique sans qu'aucun des fichiers listés dans le tableau récapitulatif (fin de document) n'ait eu à changer.

6 Calcul déterministe des dates (`rag/calendrier.py`)

Ce qui est fait

Répondre à « *puis-je chasser le sanglier le 21 janvier ?* » demande de croiser sept conditions (espèce → période d'ouverture → jour de la semaine → jours fériés → exception du 15 octobre → statut

touriste/résident → date du jour). Ce calcul est fait en **Python pur** (`chasse_autorisee`), pas par le LLM.

Pourquoi ne pas laisser le LLM calculer : un LLM ne fait pas cette chaîne de conditions de façon fiable — il produit une approximation plausible, ce qui est **pire qu'un refus**, car l'utilisateur ne peut pas distinguer une réponse juste d'une réponse fausse. Le commentaire du code cite un cas observé : le modèle avait écrit « *demain est un mercredi* » pour un mardi, en inventant au passage une histoire d'horaire — la conclusion était juste, mais par un raisonnement faux, indétectable pour l'utilisateur.

Le LLM reçoit le verdict déjà calculé (`[VERDICT CALCULÉ]` dans le prompt, `generator.py:180-181`) et se contente de le reformuler naturellement — règle 6 du prompt système lui interdit explicitement de recalculer quoi que ce soit.

Ce que ce module ne couvre pas (assumé, documenté dans le code)

- Les ~800 réserves de chasse de l'article 12 (trop de variantes d'orthographe pour une correspondance fiable)
- Les fêtes religieuses (calendrier lunaire, non calculable par date civile)

Dans les deux cas, le système le **signale** à l'utilisateur plutôt que de deviner (**reserves** dans le verdict).

7 Génération (rag/generator.py)

Ce qui est fait

- **LLM** : Groq (`config.GROQ_MODEL = "openai/gpt-oss-120b"`), appelé via l'API officielle `groq (appeler_groq)`.
- **Prompt système** (`prompt_systeme`) : 11 règles numérotées — fidélité stricte aux extraits fournis, citation d'article obligatoire, prise en compte de la validité temporelle d'un arrêté annuel, hiérarchie des normes (loi > arrêté sauf dérogation légale), priorité aux avertissements de concordance, interdiction de recalculer une date si un verdict est fourni, interdiction d'exposer le jargon interne à l'utilisateur, réponse courte pour les politesses et le hors-sujet.
- **Température basse** (`LLM_TEMPERATURE = 0.2`) : privilégie la fidélité au texte source plutôt que la créativité.
- **Contexte étiqueté** (`construire_contexte`) : chaque extrait porte sa source, son article, son rang normatif et sa validité — cet étiquetage permet au LLM d'appliquer les règles de citation et de hiérarchie des normes.

Choix du fournisseur LLM : comparaison

Fournisseur	Latence typique	Coût	Confidentialité	Pourquoi retenu / écarté
Groq (retenu)	~ 1 s / 300 mots (LPU dédié)	Faible / gratuit selon quota	Tiers	Seul fournisseur testé offrant une latence compatible avec un usage sur le terrain
Modèle local (Llama/Mistral, CPU)	1-2 min	Nul	Totale	Latence rédhibitoire pour le terrain ou une démonstration en direct
OpenAI (GPT-4o-mini)	Quelques secondes	Payant par token	Tiers	Alternative viable, non retenue faute de nécessité
Modèle local sur GPU dédié	Rapide	Coût matériel élevé	Totale	Hors

Le risque assumé de cette dépendance réseau est couvert par le cache de réponses (section suivante), pas par un repli vers un modèle local — jugé trop lent pour être un vrai filet de sécurité.

8 Cache de réponses (rag/cache.py)

- Dictionnaire JSON persistant sur disque, clé = `hash(question normalisée + spécialité)`.
- **Pas appliqué** si la question a un historique (la réponse dépend du contexte de conversation) ni si elle porte sur une date relative (`calendrier.detecter_date`) — « *demain* » ne désigne pas le même jour d'un jour à l'autre (`app/main.py:132`).
- **Usage recommandé** documenté dans le code : poser une fois chaque question de démonstration avant la soutenance, pour que celle-ci fonctionne même hors ligne.

C'est un filet de sécurité réseau, **pas** un second modèle ni une forme d'apprentissage — un simple dictionnaire clé → réponse.

9 Ce qui manque : métriques d'évaluation du RAG

Aucune métrique quantitative n'est actuellement implémentée dans le code. `scripts/test_retrieval.py` n'affiche que les chunks retrouvés pour 8 questions, à inspecter à l'œil (`Retriever.expliquer`) — c'est une vérification qualitative manuelle, pas une évaluation chiffrée, répétable et comparable entre deux versions du système.

C'est le point faible identifié pour un rapport de Master Data Science : un jury s'attend à des métriques, pas seulement à « *ça marche sur mes exemples* ».

9.1 Métriques de retrieval

Évaluent les étapes 4a-4c seules, sans appeler le LLM. Nécessitent un **jeu de questions annotées** : pour chaque question, la liste des `id` de chunks qui devraient être retrouvés (le *gold standard*), construit à la main à partir du corpus existant.

Métrique	Ce qu'elle mesure	Formule / principe
Precision@k	Parmi les k chunks retournés, combien sont réellement pertinents	chunks pertinents retrouvés / k
Recall@k	Parmi tous les chunks pertinents qui existent, combien sont dans le top-k	chunks pertinents retrouvés / total chunks pertinents
MRR (Mean Reciprocal Rank)	À quel rang apparaît le premier chunk pertinent, en moyenne	moyenne(1 / rang du premier chunk pertinent)
nDCG@k	Comme Recall@k, mais pénalise un chunk pertinent trouvé en rang 8 plus qu'en rang 1	Somme pondérée par position, normalisée

Ces métriques permettraient de répondre précisément à des questions comme : « *la fusion RRF améliore-t-elle vraiment le retrieval par rapport au dense seul ou au BM25 seul ?* » — actuellement affirmé par le code mais jamais mesuré.

9.2 Métriques de génération

Évaluent la réponse finale du LLM. Le framework **RAGAS** (bibliothèque Python dédiée à l'évaluation de systèmes RAG) formalise ces métriques, calculables sans jeu de réponses de référence — un second LLM juge la sortie du premier.

Métrique	Ce qu'elle mesure
Faithfulness (fidélité)	La réponse s'appuie-t-elle uniquement sur les extraits fournis, ou invente-t-elle des faits ? — mesure directement la règle 1 du prompt système
Answer relevancy	La réponse répond-elle réellement à la question posée ?
Context precision	Les extraits utilisés dans la réponse sont-ils parmi les plus pertinents du contexte fourni ?
Context recall	Le contexte fourni contenait-il assez d'information pour répondre complètement ?

9.3 Évaluation de bout en bout

- **Exact match / évaluation humaine** sur un jeu de ~20-30 questions représentatives (couvrant les cas déjà listés dans `test_retrieval.py`, plus des questions à verdict calendaire), avec une réponse de référence rédigée à la main.
- **Latence** : déjà partiellement mesurée (`duree_ms` dans `ChatResponse`), à agréger en moyenne/percentile 95 sur un échantillon de requêtes pour objectiver l'affirmation « *~1 seconde par réponse* ».

9.4 Ce qu'il faudrait construire avant de coder les métriques

1. Un **jeu de questions annotées** (fichier JSON, ~20-30 entrées) avec : la question, les `id` de chunks attendus (section 9.1), une réponse de référence (sections 9.2/9.3).
2. Un script `scripts/evaluer.py` qui charge ce jeu, appelle `Retriever.rechercher()` (et éventuellement `generer()`) pour chaque question, calcule les métriques et affiche un tableau récapitulatif.
3. Idéalement, ce script tourne **sans dépendre de Groq** pour la partie retrieval (9.1) — seule la partie génération (9.2/9.3) a besoin d'un appel LLM (soit le même via Groq, soit un second modèle juge).

Résumé des fichiers cités

Fichier	Rôle
rag/config.py	Tous les réglages centralisés (modèle, tailles, top-k, domaines, spécialités)
rag/chunker.py	Étape 1 — découpage du corpus en chunks
rag/indexer.py	Étapes 2 et 3 — embedding + index BM25
rag/store.py	Magasin de vecteurs (recherche dense) et application du filtre de domaine côté dense
rag/synonymes.py	Expansion de requête pour BM25
rag/retriever.py	Étape 4 — recherche hybride + fusion RRF et application du filtre de domaine côté BM25
rag/calendrier.py	Calcul déterministe des dates de chasse
rag/generator.py	Étape 5 — prompt système + appel Groq
rag/cache.py	Étape 6 — cache de réponses sur disque
app/main.py	API HTTP (POST /api/chat/), orchestration complète, calcul de domaines_pour(specialite)
app/schemas.py	Contrats de requête/réponse (Pydantic)
scripts/build_index.py	Orchestration du job d'indexation hors-ligne
scripts/test_retrieval.py	Vérification qualitative manuelle (existant)
scripts/diagnostic.py	Inspection de l'index (santé, pas performance)